



西安电子科技大学  
XIDIAN UNIVERSITY

# RAGVERUS: Repository-Level Program Verification with LLMs using Retrieval Augmented Generation

Si Cheng Zhong<sup>1</sup>, Jiading Zhu<sup>1†</sup>, Yifang Tian<sup>1†</sup>, and Xujie Si<sup>1</sup>

University of Toronto, Canada

sicheng.zhong@mail.utoronto.ca, six@cs.toronto.edu

汇报人：吴文杰  
2025年10月23日



## 现有方法的痛点

- 功能级验证局限：现有 *LLM* 验证方法（如 *AutoVerus*）聚焦单函数，依赖微调嵌入编码器，缺乏代码语法、项目结构的精准领域知识。
- 基准数据缺口：现有数据集（如 *VerusBench*）为小规模孤立场景，无仓库级复杂依赖验证基准，无法反映工业界真实需求。
- 核心挑战：仓库级验证需解决 海量语义上下文与正确依赖前提识别两大问题，*LLM* 生成证明时难以在上下文窗口内筛选最小必要引理集。



# RagVerus框架

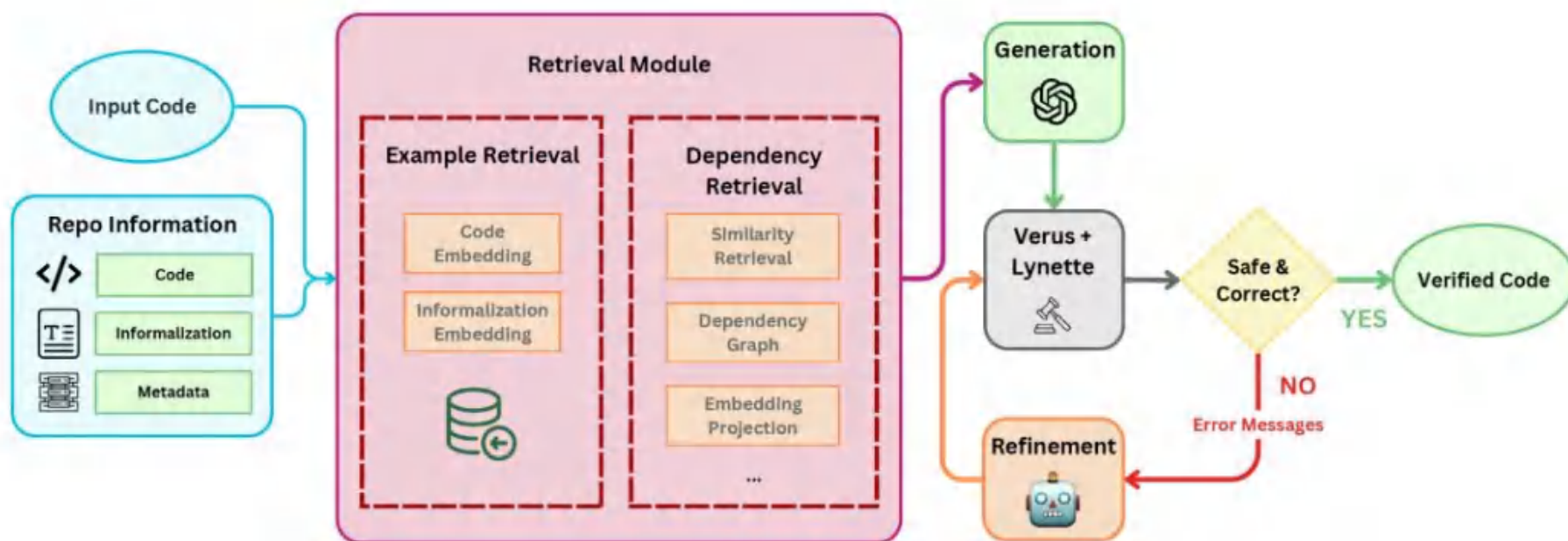


Fig. 1: Illustration of the RAGVERUS Framework Pipeline



## 核心步骤

### 1) 挖掘代码属性;

- 对仓库级代码进行静态分析, 提取函数 / 类型签名、方法调用、模块依赖等验证关键元数据与关系信息。
- 用 **OpenAI** 的 **text-embedding-3-large** 模型将代码或元数据编码为向量, 存储于 **FAISS** 构建的持久化向量库存储中, 保留语法与语义模式, 为后续检索打基础。

### 2) 检索特定任务的信息;

- 少样本示例检索: 借 **FAISS** 和 **LlamaIndex**, 从“代码体嵌入索引”“代码非形式化描述嵌入索引”中, 检索语义相似的“示例 - 证明”对, 最终筛选最多 **3** 个兼顾相关性与多样性的示例, 引导模型生成规范证明。
- 依赖检索: 通过嵌入相似性检索、微调投影匹配、依赖图 (静态分析构建代码库依赖图, 遍历调用层级提取依赖) 三种方法, 识别验证所需的依赖前提 (如函数签名)

### 3) 最终生成证明或可验证的代码。

- 接入 **AutoVerus** 等代码生成代理, 接收含可执行代码与 **Verus** 规约 (前置 / 后置条件) 的 **Rust** 模块, 结合检索到的上下文, 让大模型生成循环不变量、断言等验证注释, 形成完整注释的 **Rust** 程序。
- 用 **Verus** 编译器验证规约是否全域成立, 失败证明会反馈错误信息, 触发送代优化 (若开启), 同时该阶段还支持代码生成与规约推理的灵活扩展。



## 生成自然语言摘要 (*Text summaries*)

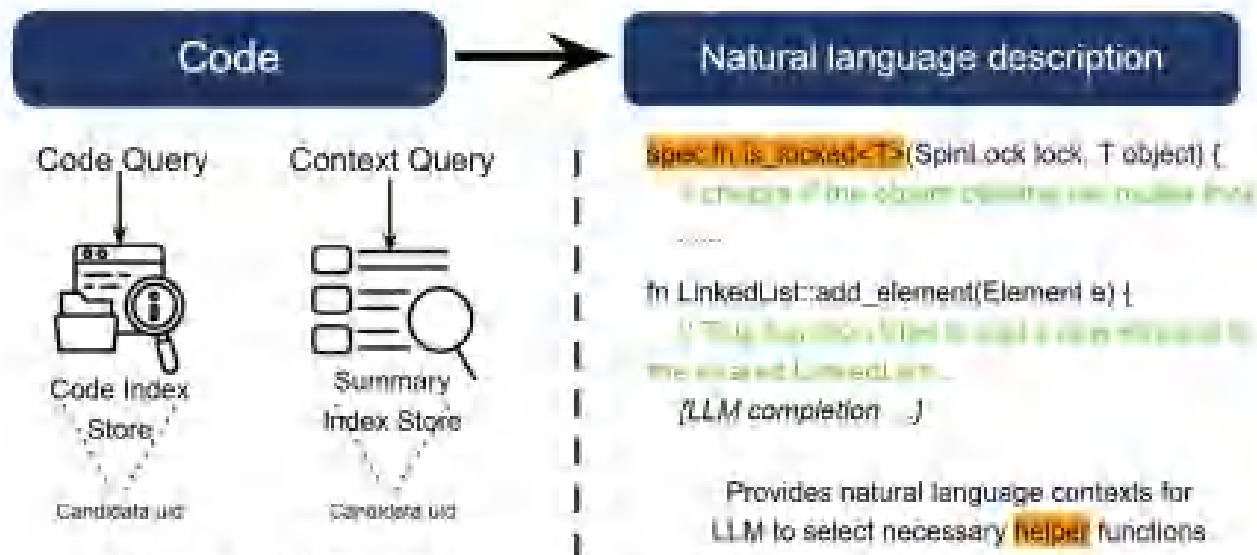


Fig. 2: Illustration of code informalization utility.

代码+*metadata*，生成函数自然语言描述，作为语义检索的另一个索引源



# Verus 属性提取器 (*Code Masking*) —— 灵活创建评估任务

- ◆ 其核心能力是识别并分离三类代码模式，通过保留关键部分、擦除目标部分生成不同类型的验证任务。

## ◆ 核心工作流程

- 代码模式识别：通过语法分析（复用 *Verus* 编译器的语法解析逻辑），扫描输入的 *Rust* 文件，自动标记出 *exec*（执行）、*spec*（规范）、*proof*（证明）三类代码的边界与内容。例如，识别 *proof fn xxx(...) { ... }* 块为证明代码，*spec fn xxx(...) -> bool { ... }* 块为规范代码。
- 任务类型定义与代码擦除：根据目标评估任务，保留“任务输入部分”，擦除“任务待解决部分”，生成查询语句（*Query*）。以最核心的“证明补全任务”为例：
  - 任务目标：评估模型能否为“有规范约束的执行代码”补全正确的证明代码。
  - 保留内容：*exec*代码（确保业务逻辑固定）和*spec*代码（确保正确性约束固定）。
  - 擦除内容：*proof*代码，用占位符（如 *// MASKED\_PROOF* 或空函数体）替代，形成“待补全的证明框架”。
- 合法性校验：擦除后自动调用 *Verus* 编译器进行语法检查，确保保留的*exec*和





## Code masking示例如下

```
impl SpinLock {
  // requires: check no deadlock
  pub fn lock(
    &self,
    Tracked(lockperm): Tracked<LockPermRaw>,
    Tracked(core): Tracked<&CoreIdPerm>,
  ) -> (perm: (Tracked<LockPermRaw>, Tracked<SnpPointsToRaw>))
  requires
    lockperm@.is_unlocked(core@.cpu, self.id(), lockperm@.points_to.range()),
  ensures
    self.ensures_lock(lockperm@, perm.0@@, perm.1@@),
  {
    let got = false;
    let mut perm: Tracked<SnpPointsToRaw>;
    let tracked mut tmplockperm = lockperm;
    loop
      invariant_except_break
        tmplockperm@.is_unlocked(core@.cpu, self.id(), lockperm@.points_to.range()),
        tmplockperm == lockperm,
      ensures
        self.ensures_lock((lockperm)@, tmplockperm@, perm@@),
      {
        if let Some(tmpperm) = self.trylock(Tracked(&mut tmplockperm), Tracked(core)) {
          perm = tmpperm;
          break ;
        }
      }
    (Tracked(tmplockperm), perm)
  }
}
```

(a) Original VerisMo Code.

```
impl SpinLock {
  // requires: check no deadlock
  pub fn lock(
    &self,
    Tracked(lockperm): Tracked<LockPermRaw>,
    Tracked(core): Tracked<&CoreIdPerm>,
  ) -> (perm: (Tracked<LockPermRaw>, Tracked<SnpPointsToRaw>))
  requires
    lockperm@.is_unlocked(core@.cpu, self.id(), lockperm@.points_to.range()),
  ensures
    self.ensures_lock(lockperm@, perm.0@@, perm.1@@),
  {
    let got = false;
    let mut perm: Tracked<SnpPointsToRaw>;
    MASKED_LINE
    loop
    MASKED_LINE
    MASKED_LINE
    MASKED_LINE
    MASKED_LINE
    {
      if let Some(tmpperm) = self.trylock(Tracked(&mut tmplockperm), Tracked(core)) {
        perm = tmpperm;
        break ;
      }
    }
    (Tracked(tmplockperm), perm)
  }
}
```

(b) Masked Code based on Verus property extractor.



# 元数据生成器（*Metadata Generation*）—— 构建任务上下文支持

- ◆ 核心作用是从 *Rust* 代码中提取验证关键元数据，为 *RagVerus* 框架的上下文检索提供结构化信息，解决仓库级验证中依赖难以定位的问题。
- ◆ 核心工作流程
  - 提取与存储：工具对 *RepoVBench* 中的 2073 个函数逐一提取元数据，生成结构化文档（如 *JSON* 格式），并与代码本身一起，通过 *OpenAI* 的 *text-embedding-3-large* 编码为向量，存入 *FAISS* 向量库。
  - 检索匹配：当 *RagVerus* 处理某验证任务时，检索模块会以任务的元数据（如函数签名、方法调用）为查询向量，在 *FAISS* 库中匹配相似元数据对应的函数，快速定位所需的依赖（如引理、被调用函数）。





## Metadata Generation示例如下

```
"spincell_e.rs-SpinLock-lock": {  
  "function_name": "lock",  
  "function_type": "Exec",  
  "implementation_name": "SpinLock",  
  "path_to_file": "lock/spincell_e.rs",  
  "start_line": 13,  
  "end_line": 40,  
  "special_types": ", Tracked<, LockPermRaw>, Tracked<LockPermRaw>, Tracked<SnpPointsToRaw>",  
  "invoked_functions": "is_unlocked, trylock, range, Tracked, lock, id, ensures_lock, Some",  
  "exist_in": "spincell_e.rs-SpinLock-lock, spincell_e.rs-<T: IsConstant + WellFormed + SpecSi",  
  "attributes": "[Impl, Signature(Exec)]",  
  "masked_line_count": 6,  
  "masked_invoked_functions": [  
    "is_unlocked",  
    "ensures_lock",  
    "id",  
    "range"  
  ],  
  "gt_func_uids": [  
    "spin_perm_s.rs-LockPermToRaw-is_unlocked",  
    "spin_perm_s.rs--is_unlocked",  
    "spin_perm_s.rs-MapRawLockTrait for LockMap-is_unlocked",  
    "spincell_e.rs-<T: IsConstant + WellFormed + SpecSize + VTypeCast<SecSeqByte>> VSpinLock",  
    "spin_t.rs-SpinLock-ensures_lock",  
    "ptr_u.rs-<V: IsConstant + WellFormed + SpecSize> SnpPointsToData<V>-id",  
    "def_s.rs-<V: IsConstant + WellFormed + SpecSize> SnpPPtr<V>-id",  
    "msr_perm_s.rs-RegisterPerm-id",  
    "vbox.rs-<T> VBox<T>-id",  
    "spin_t.rs-SpinLock-id",  
    "raw_ptr_s.rs-SnpPointsToBytes-range",  
    "param_e.rs-e!-range",  
    "range_set.rs--range"  
  ],  
},
```



## 实验评估（函数级基准）

**Table 1:** Evaluation results for VerusBench (MBPP, Diffy, Misc, and All tasks) with percentages based on total number of tasks in each category.

| Task         | Model            | Correct (n, %)    | Safe (n, %) | Success (n, %)    |
|--------------|------------------|-------------------|-------------|-------------------|
| <b>MBPP</b>  | Baseline         | 23 (29.5%)        | 60 (76.9%)  | 17 (21.8%)        |
|              | RAG VERUS - Code | 57 (73.1%)        | 74 (94.9%)  | 54 (69.2%)        |
|              | RAG VERUS - Text | 49 (62.8%)        | 68 (87.2%)  | 45 (57.7%)        |
| <b>Diffy</b> | Baseline         | 3 (7.9%)          | 30 (78.9%)  | 2 (5.3%)          |
|              | RAG VERUS - Code | 19 (50.0%)        | 38 (100.0%) | 19 (50.0%)        |
|              | RAG VERUS - Text | 24 (63.2%)        | 37 (97.4%)  | 23 (60.5%)        |
| <b>Misc</b>  | Baseline         | 7 (30.4%)         | 21 (91.3%)  | 6 (26.1%)         |
|              | RAG VERUS - Code | 11 (47.8%)        | 22 (95.7%)  | 11 (47.8%)        |
|              | RAG VERUS - Text | 8 (34.8%)         | 22 (95.7%)  | 8 (34.8%)         |
| <b>All</b>   | Baseline         | 33 (23.7%)        | 111 (79.9%) | 25 (18.0%)        |
|              | RAG VERUS - Code | <b>87 (62.6%)</b> | 134 (96.4%) | <b>84 (60.4%)</b> |
|              | RAG VERUS - Text | <b>81 (58.3%)</b> | 127 (91.4%) | <b>76 (54.7%)</b> |

注: *RagVerus-Code*表示基于代码索引的检索结果, 而*RagVerus-Text*则基于非正式化索引进行检索。



## 实验评估（仓库级基准）

**Table 2:** Evaluation results on RepoVBench (VeriSMo-Simple+Complex).

| Task                 | Model                  | Correct (n, %)    | Safe (n, %)        | Success (n, %)    |
|----------------------|------------------------|-------------------|--------------------|-------------------|
| Simple<br>52 tasks   | DirectGen greedy       | 2 (3.8%)          | <b>52 (100.0%)</b> | 2 (3.8%)          |
|                      | DirectGen sample       | 4 (7.7%)          | 48 (92.3%)         | 4 (7.7%)          |
|                      | Refinement (AUTOVERUS) | 8 (15.4%)         | 49 (94.2%)         | 7 (13.5%)         |
|                      | DirectRAG              | 21 (40.4%)        | <b>52 (100.0%)</b> | 21 (40.4%)        |
|                      | <b>Refinement+RAG</b>  | <b>24 (46.2%)</b> | 45 (86.5%)         | <b>23 (44.2%)</b> |
| Overall<br>383 tasks | Refinement (AUTOVERUS) | 63 (16.4%)        | <b>294 (76.8%)</b> | 59 (15.4%)        |
|                      | DirectRAG              | 78 (20.4%)        | 281 (73.4%)        | 65 (17.0%)        |
|                      | <b>Refinement+RAG</b>  | <b>84 (21.9%)</b> | 266 (69.5%)        | <b>75 (19.6%)</b> |

数据集规模：包含覆盖 **52** 个模块、存在依赖关系的 **2073** 个函数，可支持 **383** 个证明完成任务的评估。

不足：在实际**331**个高难度案例中，*Refinement+RAG*组合方案（**75-23=52**）仅解决了不到**16%**的验证任务。



# 总结

**RagVerus** 的核心贡献在于：首次将 **RAG** 技术系统性引入仓库级程序验证，通过“检索增强上下文”解决了 **LLM** 在大规模代码库中的依赖缺失问题，并构建了首个仓库级基准 **RepoVBench**。

## 挑战与未来方向

1. 复杂依赖场景的检索精度不足：**RepoVBench** 中 331 个 **Complex** 任务（需跨模块依赖）的解决率仍低于 16%，核心原因是现有检索方法难以精准定位关键依赖——例如部分任务需要的前提池 超出检索模块的返回上限
2. 长证明任务的生成能力有限：对于需 80 行以上证明注释的任务，现有流程因采样预算限制和迭代精修次数不足难以完成——**LLM** 生成长文本时易出现逻辑断裂，而检索模块无法动态调整上下文窗口以提供分阶段的依赖支持，这需要结合 分块生成 + 增量检索 等策略优化。
3. 基准覆盖度与泛化性待提升：当前 **RepoVBench** 仅基于 **VeriSMo** 项目构建，虽计划纳入 **Anvil** 等其他 **Verus** 项目，但整体覆盖的项目类型验证场景仍较单一。若要证明 **RagVerus** 的泛化性，需扩展至更多领域（如分布式系统、嵌入式软件）的仓库级验证任务。